

UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA



CURSO:

Algoritmos y Estructura de Datos

INFORME N°1:

Tema: Algoritmos Greedy

INTEGRANTES DEL GRUPO:

OYARCE ARANDA, Carlos Steve	20264154G
DESPOSORIO MOSQUERA, Jesus Alberto	20262140I
ÁLVAREZ ZENTENO, Max David	20264067G

DOCENTE:

SELIS VASQUEZ, Luis Antonio

FECHA DE REALIZACIÓN:

01/10/2026

FECHA DE ENTREGA:

05/10/2026

1. CONCEPTO DE ALGORITMOS GREEDY:

Los algoritmos greedy, o voraces, son métodos de resolución de problemas que, en cada paso, toman la decisión que parece mejor en ese momento. Su objetivo es que esa secuencia de decisiones locales conduzca a una solución óptima global. Se caracterizan por avanzar sin retroceder ni revisar las decisiones anteriores.

Es importante señalar que no todo problema puede resolverse de forma óptima con un algoritmo greedy; esto depende de la estructura matemática del problema.

ALGORITMOS FAMOSOS QUE SON GREEDY:

- **Dijkstra:** encuentra el camino más corto desde un nodo a los demás en grafos con pesos no negativos.
- **Kruskal:** construye un árbol de expansión mínima seleccionando primero las aristas de menor peso.
- **Prim:** también encuentra un árbol de expansión mínima, pero va expandiendo el árbol desde un nodo inicial.
- **Huffman:** se usa para compresión de datos, construyendo códigos de longitud variable según la frecuencia de los símbolos.
- **Activity Selection o selección de actividades:** elige el mayor número posible de actividades que no se solapan.
- **Fractional Knapsack o mochila fraccionaria:** selecciona objetos según la mejor relación valor/peso y permite tomar fracciones de ellos.
- **Job Sequencing with Deadlines:** organiza trabajos con fechas límite buscando maximizar el beneficio.
- **Coin Change Greedy:** intenta dar cambio usando primero las monedas de mayor valor; funciona óptimamente sólo para ciertos sistemas monetarios.

CARACTERÍSTICAS DE ESTA ESTRATEGIA:

- **Elección local óptima:** en cada paso selecciona la opción que parece mejor en ese momento.
- **Decisiones irreversibles:** una vez tomada una decisión, normalmente no se modifica ni se retrocede.
- **Construcción incremental:** la solución se forma paso a paso.
- **Criterio de selección definido:** necesita una regla clara para decidir cuál es la "mejor" opción en cada etapa.
- **Factibilidad:** cada elección debe mantener válida la solución parcial.
- **Búsqueda de un óptimo global:** pretende que la suma de buenas decisiones locales conduzca a la mejor solución total.
- **Simplicidad y eficiencia:** suelen ser algoritmos rápidos y fáciles de implementar.
- **No siempre garantizan el óptimo:** solo funcionan de forma óptima en problemas que poseen propiedades adecuadas, como la **propiedad de elección greedy** y la **subestructura óptima**.

En síntesis los algoritmos Greedy son métodos de resolución de problemas que toman en cada paso la decisión que parece mejor en ese momento, con la expectativa de que esas decisiones sucesivas conduzcan a una buena solución global.

2. ESTRATEGIA VORAZ:

El enfoque general de los algoritmos Greedy consiste en construir una solución de forma progresiva, tomando una decisión en cada paso.

La idea es la siguiente: al inicio no se tiene una solución completa. El algoritmo analiza las opciones disponibles y selecciona aquella que, según un criterio determinado, parece ser la mejor decisión en ese momento. Esa elección se incorpora a la solución parcial y luego se continúa con el problema restante.

El proceso se repite así:

1. Se parte de una solución vacía o inicial.
2. Se identifican las opciones disponibles.
3. Se elige la opción localmente más conveniente.
4. Se verifica que esa elección sea válida o factible.
5. Se incorpora a la solución.
6. Se continúa con las opciones restantes hasta completar el problema.

Por eso se dice que la solución se construye mediante decisiones sucesivas:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_n$$

Donde cada S_i representa una solución parcial obtenida después de una nueva elección.

Por ejemplo, si se quiere seleccionar actividades que no se superpongan, una estrategia Greedy puede elegir primero la actividad que termina más temprano. Luego, entre las actividades restantes compatibles, vuelve a escoger la que termina más temprano, y así sucesivamente.

El punto clave es que una estrategia Greedy no analiza todas las combinaciones posibles. Esto la hace generalmente rápida y sencilla, aunque solo en ciertos problemas puede demostrarse que esas decisiones locales producen también la solución óptima global.

3. ELECCIÓN LOCAL:

La esencia de un algoritmo voraz radica en su mecanismo de toma de decisiones, el cual se define a través del concepto de “elección local”, palabra muy importante para el desarrollo de algoritmos Greedy. El análisis de cómo se selecciona la mejor alternativa disponible en cada etapa del algoritmo revela que el sistema evalúa un conjunto de candidatos basándose exclusivamente en el beneficio inmediato. Esto se logra mediante el uso de una función de selección que asigna un valor a cada alternativa presente. En cada iteración, el algoritmo analiza un conjunto de candidatos (posibles alternativas) y aplica un criterio de selección. Este criterio evalúa el beneficio inmediato de cada opción de forma aislada, es decir, el

algoritmo escogerá aquella que maximice la ganancia o minimice el costo en ese instante preciso, ignorando por completo las posibles repercusiones futuras de dicha decisión.

Este enfoque puramente local dota al algoritmo de características muy particulares. En primer lugar, introduce lo que se conoce como una "miopía estructural", que tiene que ver con lo que dije antes, ya que, al tomar la decisión, el algoritmo carece de visión a largo plazo; no examina el espacio completo de soluciones ni considera cómo una elección aparentemente óptima ahora podría forzar decisiones desfavorables más adelante. En segundo lugar, las decisiones tomadas bajo la elección local son estrictamente irrevocables. A diferencia de paradigmas como el Backtracking (vuelta atrás), un algoritmo greedy nunca deshace una elección. Una vez que un elemento se añade a la solución parcial, permanece allí hasta el final de la ejecución.

4. PROPIEDAD DE ELECCIÓN LOCAL:

Si bien el mecanismo de elección local es rápido y eficiente, no siempre garantiza encontrar la solución globalmente óptima al problema. Para que un algoritmo voraz sea matemáticamente correcto y halle el óptimo global, el problema a resolver debe cumplir imperativamente con una condición conocida como la "Propiedad de elección voraz". Esta propiedad establece que una solución global óptima puede alcanzarse mediante la toma sucesiva de decisiones óptimas locales, es decir, nos asegura que tomar la decisión más golosa o jugosa, por así llamarla ahora mismo, no va a arruinar nuestros resultados a largo plazo.

La explicación de las condiciones que permiten que una elección local forme parte de una solución óptima se fundamenta en la independencia de las decisiones y la reducción estructural del problema. Cuando un algoritmo realiza su primera elección local (la cual parece ser la mejor en ese momento), el problema original se simplifica y se reduce a un único subproblema restante. La condición crucial aquí es que la resolución de este subproblema restante no debe requerir que revisemos o alteremos la decisión inicial.

Dicho de otro modo, una elección local es segura si se puede demostrar que existe al menos una solución óptima del problema completo que contiene dicha elección. En el análisis formal de algoritmos, esto suele justificarse mediante un método conocido como el "argumento de intercambio" (exchange argument). Este proceso lógico consiste en asumir hipotéticamente una solución óptima que no incluya la elección greedy inicial; luego, se demuestra que al sustituir un elemento de esa solución hipotética por nuestra elección greedy, el valor de la solución resultante no empeora. Al probar esto, se confirma que la estrategia voraz siempre nos mantendrá en el camino hacia la perfección global.

Un contraste claro se observa en los problemas de la mochila (Knapsack problem). En la versión **fraccional** del problema, se cumple la propiedad de elección voraz, ya que siempre conviene llevarse primero el objeto con mayor relación valor/peso. Sin embargo, en la versión 0-1 (donde no se pueden fraccionar los objetos), la propiedad se rompe. En este último caso, la elección miope de tomar el objeto más valioso podría agotar la capacidad de la mochila, impidiendo cargar una combinación de objetos más ligeros que, en conjunto, sumarían un valor mayor. Esto demuestra que la **validación de la propiedad de elección voraz** es un paso obligatorio antes de implementar este paradigma.

Ejemplo: El cambio de monedas.

Un ejemplo clásico de la vida diaria para explicar la elección voraz es el problema de dar cambio. Imagina que trabajas en una tienda y debes dar 67 céntimos de cambio usando monedas de 25, 10, 5 y 1. Quieres usar la menor cantidad de monedas posible.

La estrategia de tu elección local en cada etapa será: "Elegir la moneda más grande que no supere lo que me falta por pagar". Veamos el proceso:

1. Te faltan 67. La mayor es 25. (Restan 42)
2. Te faltan 42. La mayor es 25. (Restan 17)
3. Te faltan 17. La mayor es 10. (Restan 7)
4. Te faltan 7. La mayor es 5. (Restan 2)
5. Te faltan 2. La mayor es 1. (Resta 1)
6. Te falta 1. La mayor es 1. (Resta 0)

Este sistema de monedas cumple perfectamente con las condiciones de la propiedad de elección voraz. Las selecciones "locales" (agarrar la moneda más grande en el momento) conformaron la solución óptima (usar solo 6 monedas).

5. CORRECCIÓN DE LA ESTRATEGIA VORAZ:

Un algoritmo es correcto cuando, para cualquier instancia válida del problema, termina y entrega una solución factible y óptima. En un algoritmo greedy esto no es evidente: como nunca explora otras alternativas ni retrocede, que una decisión "parezca buena" no prueba nada. Por eso la corrección de una estrategia voraz debe demostrarse: hay que justificar por qué elegir lo mejor en cada paso no cierra el camino hacia el óptimo global.

Condiciones que garantizan la corrección

Una elección local conduce a una solución óptima cuando el problema cumple dos propiedades al mismo tiempo:

1. **Propiedad de elección voraz:** existe al menos una solución óptima que contiene la elección que hace el algoritmo (la elección es "segura").
2. **Subestructura óptima:** una vez tomada esa elección, lo que queda es un subproblema del mismo tipo, y su solución óptima, unida a la elección hecha, forma una solución óptima del problema completo (se desarrolla en la sección 6).

Si ambas se cumplen, la corrección se concluye por inducción sobre el tamaño del problema:

- a. **Caso base:** si no quedan elementos por decidir, la solución vacía es óptima.
- b. **Paso inductivo:** sea P el problema y g la elección greedy. Por la propiedad de elección voraz, existe una solución óptima O que contiene a g . Por la subestructura óptima, $O \setminus \{g\}$ es óptima para el subproblema P_g que queda después de elegir g . Por

hipótesis de inducción, el algoritmo resuelve P_g de forma óptima; luego, g más la solución que el algoritmo entrega para P_g es óptima para P .

Técnicas para demostrar la corrección

a) Argumentos de intercambio (exchange argument).

1. Se toma una solución óptima cualquiera O .
2. Si O ya contiene la elección greedy g , no hay nada que probar.
3. Si no la contiene, se identifica el elemento x de O que “compite” con g y se construye $O' = (O \setminus \{x\}) \cup \{g\}$
4. Se demuestra que O' es factible y que su valor no es peor que el de O . Como O era óptima, O' también lo es y contiene a g .

b) El greedy se mantiene adelante (greedy stays ahead).

Se compara paso a paso la solución del algoritmo $G = (g_1, g_2, \dots, g_k)$ con cualquier otra solución $O = (o_1, o_2, \dots, o_m)$ usando una medida de progreso, y se demuestra que después de cada paso G va al menos tan bien como O . Al final se concluye que G no puede ser peor.

c) Contraejemplo (para refutar).

Para demostrar que un criterio greedy no es correcto basta con encontrar una sola instancia donde falle. Es la forma más rápida de descartar un criterio que “parece lógico”.

Cuando la elección local NO conduce al óptimo

- **Selección de actividades por menor duración.** Con $A=[1,5)$, $B=[4,7)$ y $C=[6,10)$, la actividad más corta es B ; al elegirla se bloquean A y C (resultado: 1 actividad) cuando lo óptimo era $\{A,C\}$ (2 actividades). El intercambio falla porque B se traslapa con dos actividades de la solución óptima, y reemplazar una sola de ellas no elimina el conflicto con la otra.
- **Selección de actividades por menor hora de inicio.** Con $A=[0,10)$, $B=[1,2)$ y $C=[3,4)$, el criterio elige A (1 actividad) y se pierden B y C (2 actividades).
- **Cambio de monedas con denominaciones {1, 3, 4} y monto 6.** Greedy toma $4+1+1$ (3 monedas), pero el óptimo es $3+3$ (2 monedas). Con sistemas como $\{1,5,10,25\}$ sí funciona.
- **Mochila 0-1 con el criterio valor/peso.** Con capacidad 50 y objetos (valor, peso) = (60, 10), (100, 20), (120, 30), el criterio toma los dos primeros (valor 160), pero el óptimo es tomar los dos últimos (valor 220).

Resumen de criterios analizados

Problema	Criterio greedy	¿Es correcto?	Cómo se justifica
Selección de actividades	Menor hora de fin	Sí	Intercambio / greedy stays ahead
	Mayor hora de inicio (de atrás hacia adelante)	Sí	Es el caso simétrico del anterior
	Menor duración	No	Contraejemplo $\{[1,5), [4,7), [6,10)\}$
	Menor hora de inicio	No	Contraejemplo $\{[0,10), [1,2), [3,4)\}$
Mochila fraccionaria	Mayor razón valor/peso	Sí	Intercambio de peso entre objetos
	Mayor valor	No	Contraejemplo (sección 7.2)
Mochila 0-1	Mayor razón valor/peso	No	Contraejemplo (60, 100, 120)
Cambio de monedas	Moneda más grande posible	Solo en sistemas canónicos	Contraejemplo $\{1,3,4\}$

6.SUBESTRUCTURA ÓPTIMA:

Se dice que un problema tiene subestructura óptima si una solución óptima del problema contiene dentro de sí las soluciones óptimas de sus subproblemas. Es decir, el problema global se puede descomponer en problemas más pequeños, y la mejor solución para el todo se construye a partir de las mejores soluciones para las partes.

De forma más precisa, sea O una solución óptima de P , sea x una de las decisiones que forman O y sea P_x el subproblema que queda después de tomar x . Hay subestructura óptima si $O \setminus \{x\}$ es una solución óptima de P_x . En términos de valores:

$$OPT(P) = valor(x) + OPT(P_x)$$

Relación con los algoritmos greedy

En un algoritmo greedy, la subestructura óptima permite reducir el problema: tras cada elección queda un solo subproblema más pequeño del mismo tipo, y se vuelve a aplicar la misma regla.

$$P \rightarrow^g P_g \rightarrow^{g'} P_{gg'} \rightarrow^{g''} \dots \rightarrow \emptyset$$

La subestructura asegura que la parte que queda por resolver sigue siendo “lo mejor posible”, y la propiedad de elección voraz asegura que la decisión que se toma ahora es segura. Con ambas, el valor óptimo se obtiene sin comparar alternativas:

$$\text{Greedy: } OPT(P) = valor(g) + OPT(P_g) \quad (\text{la elección } g \text{ ya se conoce})$$

$$\begin{aligned} \text{Prog. dinámica: } OPT(P) \\ = \max_{x \in E(P)} \{valor(x) + OPT(P_x)\} \quad (\text{hay que probar todas las elecciones } x) \end{aligned}$$

Donde $E(P)$ es el conjunto de elecciones posibles en P . La diferencia entre ambas fórmulas es exactamente el operador $\max$: el greedy lo elimina porque sabe de antemano cuál elección es segura. Por eso es tan rápido, y por eso solo funciona cuando esa elección puede demostrarse.

Característica	Fuerza bruta	Algoritmo greedy	Programación dinámica
Necesita subestructura óptima	No	Sí	Sí
Necesita propiedad de elección voraz	No	Sí	No
Elecciones evaluadas en cada paso	Todas	Una sola	Todas (guardando resultados)
Subproblemas que quedan tras decidir	Muchos	Uno	Varios (con superposición)
Garantiza el óptimo	Siempre	Solo si cumple ambas propiedades	Siempre que haya subestructura
Costo típico	Exponencial	Bajo (lineal o $n \log n$)	Polinomial (mayor que el greedy)

La subestructura óptima es necesaria pero no suficiente para que un algoritmo greedy sea correcto. Si el problema solo tiene subestructura óptima, se resuelve con programación dinámica; si además cumple la propiedad de elección voraz, alcanza con un algoritmo greedy.

Ejemplos

1. **Selección de actividades (tiene ambas propiedades).** Sea S_1 el conjunto de actividades que empiezan en f_1 o después. Si O es óptima y contiene a a_1 (la que termina primero), entonces $O \setminus \{a_1\}$ es una solución óptima de S_1 ; si no lo fuera, se podría reemplazar por una mejor y O no sería óptima (cortar y pegar). Esto lleva a la relación:

$$|A(S)| = 1 + |A(S_1)|$$

Donde $A(S)$ es un conjunto máximo de actividades compatibles dentro de S .

2. **Mochila fraccionaria (tiene ambas propiedades).** Si se toma completo el objeto 1 (el de mayor razón valor/peso), queda el mismo problema con capacidad $W - w_1$ y sin ese objeto:

$$V(W, I) = v_1 + V(W - w_1, I \setminus \{1\})$$

Si lo que quedara de la mochila fuera mejorable, también lo sería la solución completa.

3. **Cambio de monedas con {1,3,4} (subestructura sin elección voraz).** Si la mejor forma de dar cambio de 6 usa una moneda de 3, lo que falta (3) debe darse también con la menor cantidad de monedas posible: la subestructura se cumple. Sin embargo, la elección greedy (la moneda de 4) no forma parte de ninguna solución óptima de 6. El problema se resuelve entonces con programación dinámica:

$$C(m) = 1 + \min_{c_i \leq m} C(m - c_i), \quad C(0) = 0$$

El mismo caso se da en la mochila 0-1: tiene subestructura óptima, pero no propiedad de elección voraz.

4. **Un problema sin subestructura óptima.** El camino simple más largo entre dos vértices de un grafo no lo tiene: un subcamino de un camino simple máximo no es necesariamente el camino simple máximo entre sus extremos. Ni la programación dinámica ni greedy pueden construir la solución a partir de las subsoluciones.

7. PROBLEMAS CLÁSICOS:

1) Problemas de cambio de monedas.

Problema: Una máquina debe devolver cambio utilizando las siguientes monedas:

$$\{50, 20, 10, 5, 2, 1\}$$

Dado un monto entero positivo, debe determinar qué monedas debe utilizar para devolver el cambio usando la **menor cantidad posible de monedas**. Requisitos, el programa debe: solicitar al usuario un monto entero positivo, utilizar las monedas $\{50, 20, 10, 5, 2, 1\}$, seleccionar siempre primero la moneda de mayor denominación posible, mostrar las monedas utilizadas, mostrar la cantidad total de monedas empleadas.

Estrategia Greedy: Elegir siempre la moneda de mayor valor posible que no supere el monto restante.

La regla Greedy se puede expresar así:

En cada paso, seleccionar la moneda más grande que sea $\leq$ monto restante

La solución se va construyendo paso a paso hasta que el monto restante sea cero.

Así se vería el problema en el terminal.

```
Problems  Output  Debug Console  Terri

Ingrese el monto: 567
11 moneda(s) de 50
1 moneda(s) de 10
1 moneda(s) de 5
1 moneda(s) de 2
Cantidad total de monedas: 14
PS C:\Users\josif\Desktop\max>
```

Características, Eficiencia e Implementación:

- **Característica:** En cada paso se escoge la moneda de mayor valor que no exceda el monto restante. Decisiones sucesivas: la solución se construye moneda por moneda. Es simple de entender e implementar: solo requiere recorrer las denominaciones de mayor a menor. Busca minimizar la cantidad de monedas: siempre intenta usar primero las monedas más grandes. No funciona óptimamente para cualquier conjunto de monedas: depende de las denominaciones disponibles.
- **Eficiencia:** Si existen n tipos de monedas y se recorren una sola vez de mayor a menor, usando división entera para saber cuántas monedas usar, la complejidad es: $O(n)$, donde n es la cantidad de denominaciones.

La memoria adicional utilizada es: $O(1)$ si solo se almacenan contadores y variables simples.

Una implementación con “while” por cada moneda puede depender también de la cantidad total de monedas entregadas, por lo que una forma más eficiente es utilizar división y módulo.

- **Implementación:** La facilidad de implementación de este algoritmo Greedy de cambio de monedas es alta, porque su lógica es muy directa. Solo necesitas: tener las monedas ordenadas de mayor a menor, luego recorrerlas una por una hasta tomar tantas monedas como sea posible de cada denominación; reducir el monto restante y continuar hasta llegar a cero.

Ventajas, Desventajas y Situaciones de uso:

- **Ventajas:** Es fácil de entender e implementar y es rápido, porque toma decisiones directas sin explorar muchas combinaciones; a su vez, usa poca memoria, ya que no necesita almacenar muchas soluciones intermedias.

Puede encontrar la solución óptima cuando las denominaciones de monedas tienen una estructura adecuada. Es útil cuando se necesita una respuesta rápida y sencilla.

- **Desventajas:** No siempre encuentra la solución óptima. Depende mucho del conjunto de monedas disponibles. No reconsidera decisiones anteriores. Puede dar una solución válida, pero con más monedas de las necesarias.
- **Situaciones de uso:** Este algoritmo se puede utilizar cuando: Las denominaciones de monedas permiten que la estrategia Greedy produzca siempre el mínimo número de monedas, se necesita calcular el cambio de manera rápida, se trabaja con máquinas expendedoras, cajas registradoras o sistemas automáticos de pago, se quiere obtener una solución aproximada rápidamente, aunque no necesariamente sea la mejor posible y cuando el problema tiene demostrada la propiedad de elección Greedy, es decir, elegir la mejor opción local conduce al óptimo global.

Análisis Matemático de Complejidad:

Complejidad Temporal: Supongamos que existen n denominaciones de monedas:

$$M = \{m_1, m_2, \dots, m_n\}$$

Ordenadas de mayor a menor. El algoritmo realiza, para cada moneda:

$$\text{cantidad} = \text{monto} / \text{monedas}[i];$$

$$\text{monto} = \text{monto} \% \text{monedas}[i];$$

y recorre todas las denominaciones con:

$$\text{for (int } i = 0; i < n; i++)$$

Por tanto, el número de iteraciones principales es:

$$T(n) = n$$

En cada iteración se realizan operaciones de tiempo constante, como división, comparación, suma y módulo. Podemos representar el costo de cada iteración mediante una constante c :

$$T(n) = c_1 + c_2 n$$

Al analizar la complejidad asintótica ignoramos las constantes y los términos de menor orden:

$$T(n) = O(n)$$

Por lo tanto, la complejidad temporal es: $O(n)$. Donde " n " es el número de tipos de monedas disponibles.

Complejidad Espacial: El algoritmo utiliza unas pocas variables: monto, cantidad, total de monedas, y un contador " i ".

Estas variables ocupan una cantidad fija de memoria que no depende del valor del monto. Por tanto, si no contamos el arreglo de denominaciones como memoria auxiliar:

$$O(1)$$

de espacio adicional.

Si consideramos que las n denominaciones se almacenan explícitamente en un arreglo:

$$\text{int monedas}[n];$$

El almacenamiento total sería: $O(n)$. Aunque la memoria auxiliar del algoritmo sigue siendo $O(1)$.

2) Selección de actividades.

Problema: Una empresa dispone de una única sala de reuniones y tiene varias actividades programadas durante el día. Cada actividad tiene una hora de inicio y una hora de finalización. Como solo existe una sala, dos actividades no pueden realizarse al mismo

tiempo. El objetivo es seleccionar la mayor cantidad posible de actividades que no se superpongan.

Estrategia Greedy: La regla que se debe utilizar es: Seleccionar siempre la actividad compatible que termine más temprano.

Después de seleccionar una actividad, solo pueden elegirse actividades cuyo tiempo de inicio sea mayor o igual al tiempo de finalización de la última actividad seleccionada:

$$inicio_{nueva} \geq fin_{última}$$

En el terminal se vería así:

```
Actividad actividades[] = {
    {'A', 1, 4},
    {'B', 3, 5},
    {'C', 0, 6},
    {'D', 5, 7},
    {'E', 3, 9},
    {'F', 5, 9},
    {'G', 6, 10},
    {'H', 8, 11},
    {'I', 8, 12},
    {'J', 2, 14},
    {'K', 12, 16}
};
```

```
Problems  Output  Debug Console  Terminal  Ports

Actividades seleccionadas:
A: 1 - 4
D: 5 - 7
H: 8 - 11
K: 12 - 16

Total de actividades seleccionadas: 4
PS C:\Users\josif\Desktop\max>
```

Características, Eficiencia e Implementación:

- **Características:** Tiene elección local óptima: en cada paso se selecciona la actividad compatible que termina más temprano. Construcción progresiva: la solución se forma agregando actividades una por una. Tampoco hay retroceso: una

actividad seleccionada no se elimina posteriormente y esto garantiza una solución óptima para este problema clásico, es decir, obtiene la mayor cantidad posible de actividades compatibles.

- **Eficiencia:** Si las actividades ya están ordenadas por hora de finalización, el algoritmo solo necesita recorrerlas una vez:

$$O(n)$$

Donde n es el número de actividades. Si primero es necesario ordenarlas, la ordenación normalmente cuesta:

$$O(n \log n)$$

y luego la selección cuesta:

$$O(n)$$

Por tanto, la complejidad total es:

$$O(n \log n) + O(n)$$

y se simplifica a:

$$O(n \log n)$$

La memoria auxiliar puede ser:

$$O(1)$$

si la selección se realiza directamente sobre el arreglo y no se almacenan copias adicionales.

- **Implementación:** La implementación es relativamente sencilla. Primero se ordenan las actividades de menor a mayor según su hora de finalización. Después:

seleccionar la primera actividad;

```
for (int i = 1; i < n; i++) {  
    if (actividades[i].inicio >= ultimoFin) {  
        seleccionar actividades[i];  
        ultimoFin = actividades[i].fin;  
    }  
}
```

La condición principal es:

`actividades[ i ].inicio >= ultimoFin`

Porque determina si la actividad actual no se superpone con la última seleccionada.

Ventajas, Desventajas y Situaciones de uso:

- **Ventajas:** Es simple de entender e implementar y es eficiente, especialmente si las actividades ya están ordenadas por tiempo de finalización.

Garantiza una solución óptima para el problema clásico de seleccionar la mayor cantidad de actividades no superpuestas, a su vez, requiere poca memoria adicional y evita explorar todas las combinaciones posibles de actividades.

- **Desventajas:** Solo funciona óptimamente cuando el objetivo es maximizar la cantidad de actividades compatibles y no sirve directamente si las actividades tienen diferentes prioridades, ganancias o costos.
También depende de ordenar correctamente las actividades por hora de finalización. Una vez tomada una decisión, el algoritmo no retrocede para reconsiderarla. Si cambia el objetivo del problema, puede ser necesario usar otra estrategia.
- **Situaciones de uso:** Es útil para organizar el uso de una sala de reuniones, programar tareas en una sola máquina o recurso, seleccionar el mayor número de clases, eventos o conferencias sin conflictos de horario también para asignar turnos que no se superpongan, organizar reservas de un recurso compartido y para programar trabajos con intervalos de tiempo definidos.

Análisis Matemático de Complejidad:

- **Complejidad Espacial:** Depende de qué memoria consideremos. Si las actividades ya están almacenadas en un arreglo y el algoritmo solo usa variables (`int ultimoFin`, `int cantidad`, `int i`)

Entonces la memoria adicional es constante:

$$O(1)$$

Esto se llama complejidad espacial auxiliar. Si además contamos el arreglo donde se almacenan las n actividades, entonces se necesita:

$$O(n)$$

De memoria total.

Por tanto:

Espacio auxiliar: $O(1)$

Espacio total considerando las actividades: $O(n)$

Para analizar el algoritmo normalmente se destaca que su complejidad espacial auxiliar es $O(1)$.

- **Complejidad Temporal:** Depende de si las actividades ya están ordenadas por su tiempo de finalización. Si ya están ordenadas, el algoritmo las recorre una sola vez:

$$O(n)$$

Donde n es el número de actividades.

En caso de que primero haya que tener que ordenarlas por tiempo de finalización, entonces:

$$O(n \log n)$$

para ordenar, más:

$$O(n)$$

Para recorrerlas y seleccionar las compatibles. Por tanto, la complejidad total es

$$O(n \log n) + O(n)$$

Y se simplifica a:

$$O(n \log n)$$

En resumen: $O(n)$ si ya están ordenadas y $O(n \log n)$ si hay que ordenarlas primero.

3) Asignación de Galletas a Niños (Assign Cookies)

Problema: Tienes un grupo de niños, cada uno con un nivel de "hambre" (un número), y tienes un grupo de galletas, cada una con un "tamaño". Un niño solo se conforma si el tamaño de la galleta es mayor o igual a su nivel de hambre. Quieres contentar a la mayor cantidad de niños.

Estrategia Greedy: Intentar contentar primero al niño menos hambriento con la galleta más pequeña que le sirva. (Asumiremos que los arreglos ya están ordenados de menor a mayor para mantener el código en nivel básico).

Caso:

- Niños (Nivel de hambre): {1, 2, 3}
- Galletas (Tamaños disponibles): {1, 1, 3}

Paso a paso de la ejecución:

1. **Niño 1 (hambre 1) vs Galleta 1 (tamaño 1):** La galleta satisface al niño ($1 \geq 1$). El niño 1 se va contento. Avanzamos al siguiente niño y a la siguiente galleta. (*Niños contentos: 1*)
2. **Niño 2 (hambre 2) vs Galleta 2 (tamaño 1):** La galleta NO satisface al niño ($1 < 2$). Como la galleta es muy pequeña, se descarta y se prueba con la siguiente galleta disponible para el mismo niño.
3. **Niño 2 (hambre 2) vs Galleta 3 (tamaño 3):** La galleta satisface al niño ($3 \geq 2$). El niño 2 se va contento. Avanzamos al siguiente niño. (*Niños contentos: 2*)
4. **Fin del proceso:** El Niño 3 (hambre 3) se queda esperando, pero como ya no hay más galletas en el arreglo, el bucle `while` termina.
5. **Ejecución del programa:**

```
Cantidad maxima de ninos contentos: 2
-----
Process exited after 0.03672 seconds with return value 0
Presione una tecla para continuar . . .
```

Características, Eficiencia e Implementación

- **Eficiencia:** El algoritmo procesa los datos mediante un único recorrido iterativo usando un ciclo `while`. Al avanzar constantemente sobre el arreglo de galletas (`j++`), el tiempo de ejecución de esta fase se mantiene estrictamente lineal.
- **Facilidad de implementación:** Altamente sencilla. Utiliza dos arreglos y un par de índices (`i` y `j`) para recorrer las estructuras. La decisión central se basa en una única condición `if` para verificar si el tamaño de la galleta satisface el nivel de hambre.
- **Características particulares:** La estrategia greedy asume como premisa que los arreglos ya están ordenados de menor a mayor para mantener la simplicidad del código. Su comportamiento particular es intentar contentar primero al niño menos hambriento con la galleta más pequeña que le sirva; e, independientemente de si la galleta cumplió el requisito o no, el índice de las galletas siempre avanza a la siguiente.

Ventajas, Desventajas y Situaciones de Uso

- **Ventajas:** Logra encontrar la solución óptima para contentar a la mayor cantidad de niños de forma muy rápida y sin consumir memoria adicional estructurada, asignando los recursos de manera que no se desperdicien galletas grandes en niños con poca hambre.
- **Desventajas:** Depende estrictamente de la precondition de que los arreglos de entrada ya vengan ordenados. Si el programa recibiera datos aleatorios, obligaría a implementar un paso previo de ordenamiento, lo que elevaría la complejidad temporal del proceso completo.
- **Cuándo resulta conveniente:** En sistemas de asignación de recursos limitados con requerimientos mínimos variables. Por ejemplo, al asignar tareas computacionales (niños/hambre) a servidores con distintas capacidades (galletas/tamaño), asegurando que el recurso de menor capacidad capaz de cumplir una tarea se utilice primero, maximizando la cantidad total de tareas procesadas.

Análisis Matemático de Complejidad

Complejidad Temporal: $O(N)$, donde N representa la longitud limitante de los arreglos (asumiendo que los arreglos ya están ordenados).

- *Qué representa:* El tiempo total de ejecución de la fase de asignación está dominado por el recorrido secuencial, el cual crece de forma directamente proporcional a la cantidad de galletas a evaluar.
- *Demostración matemática mediante límites:* El núcleo del algoritmo es un bucle `while` que continúa iterando mientras haya niños (`i < num_ninos`) y galletas disponibles (`j < num_galletas`). Dentro de este ciclo, el índice `j` se incrementa en todas y cada una de las iteraciones (`j++`), independientemente de si la evaluación condicional es verdadera o falsa. Esto garantiza que el bucle iterará, como máximo, un número de veces igual a `num_galletas`. Si el conjunto de operaciones internas y evaluaciones toma un tiempo constante c , la función de tiempo es:

$$T(n) = c \cdot n$$

- Para evaluar el orden de complejidad, calculamos el límite respecto a n cuando tiende a infinito:

$$\lim_{n \rightarrow \infty} \frac{c \cdot n}{n} = c$$

- Al obtener una constante estricta mayor que cero (c), se demuestra asintóticamente que la función del bloque evaluado tiene un ritmo de $O(N)$. (Nota: Si se considerara un algoritmo de ordenamiento previo en lugar de asumirlo, la complejidad global subiría a $O(n \log n)$).

Complejidad Espacial: $O(1)$.

- *Qué representa:* El espacio en memoria extra que utiliza el programa, sin contar los arreglos originales de hambre y tamaño de galletas.
- *Demostración:* El programa únicamente requiere de variables de tipo primitivo para operar: contadores de capacidad (`num_ninos`, `num_galletas`), la variable de resultado (`ninos_contentos`) y los iteradores (`i`, `j`). Al no instanciar memoria dinámica adicional ni recurrir a pilas de llamadas recursivas, el espacio adicional requerido se mantiene constante, comprobando que es de orden $O(1)$.

4) Comprar la mayor cantidad de artículos.

Problema: Tienes un presupuesto y una lista de precios. ¿Cuál es la mayor cantidad de artículos que puedes comprar?

Estrategia greedy: Comprar siempre el artículo más barato que quede. Los precios ya vienen ordenados de menor a mayor.

Caso:

- Precios de los artículos ordenados: `{2, 3, 5, 8, 12}`
- Presupuesto inicial: `20`

Paso a paso de la ejecución:

1. **Artículo 1 (precio 2):** El presupuesto de 20 es suficiente ($20 \geq 2$). Se compra. Presupuesto restante: $20 - 2 = 18$. (*Cantidad acumulada: 1*)
2. **Artículo 2 (precio 3):** El presupuesto de 18 es suficiente ($18 \geq 3$). Se compra. Presupuesto restante: $18 - 3 = 15$. (*Cantidad acumulada: 2*)
3. **Artículo 3 (precio 5):** El presupuesto de 15 es suficiente ($15 \geq 5$). Se compra. Presupuesto restante: $15 - 5 = 10$. (*Cantidad acumulada: 3*)
4. **Artículo 4 (precio 8):** El presupuesto de 10 es suficiente ($10 \geq 8$). Se compra. Presupuesto restante: $10 - 8 = 2$. (*Cantidad acumulada: 4*)
5. **Artículo 5 (precio 12):** El presupuesto de 2 NO es suficiente ($2 < 12$). Se ejecuta la condición `else` y el `break` detiene el ciclo de evaluación.

6. Ejecución del programa:

```
Articulos comprados (precios): 2 3 5 8
Cantidad = 4, sobrante = 2

-----
Process exited after 0.03961 seconds with return value 0
Presione una tecla para continuar . . .
```

Características, Eficiencia e Implementación

- **Eficiencia:** Es extremadamente eficiente, ya que la evaluación se realiza en una sola pasada lineal sobre el arreglo. El algoritmo se detiene inmediatamente usando la instrucción **break** en cuanto el presupuesto no alcanza para el artículo actual, evitando iteraciones innecesarias.
- **Facilidad de implementación:** Su codificación es elemental y directa. Solo requiere inicializar variables básicas, un único bucle **for** para recorrer el arreglo de precios, y una condición **if-else** para restar el precio al presupuesto e incrementar el contador de artículos, o en su defecto salir del ciclo.
- **Características particulares:** Depende completamente de que los datos de entrada ya estén preprocesados, dado que asume que el arreglo de precios está explícitamente ordenado de menor a mayor desde el inicio. Su decisión local óptima se fundamenta en elegir secuencialmente el elemento más barato disponible.

Ventajas, Desventajas y Situaciones de Uso

- **Ventajas:** Encuentra la solución global óptima para maximizar la cantidad de elementos seleccionados de forma muy rápida y con un consumo nulo de memoria adicional estructurada.
- **Desventajas:** Asume obligatoriamente que el arreglo ya se encuentra ordenado. Si el sistema recibiera datos de precios desordenados, habría que aplicar un algoritmo de ordenamiento previo, lo que aumentaría la complejidad temporal dominante de todo el proceso a $O(n \log n)$. Adicionalmente, el algoritmo solo maximiza la cantidad de unidades, pero no busca la combinación exacta que minimice el sobrante del presupuesto.
- **Cuándo resulta conveniente:** Es ideal en escenarios donde el objetivo principal es el volumen, procesando la mayor cantidad de tareas o adquiriendo la mayor cantidad de bienes de bajo costo posible con un recurso limitante. Ejemplos prácticos incluyen asignar el mayor número de tareas computacionales ligeras a un servidor con tiempo limitado o realizar compras por volumen donde todas las unidades aportan el mismo valor.

Análisis Matemático de Complejidad

Complejidad Temporal: $O(n)$, donde n es la cantidad de artículos en el arreglo.

- a) *Qué representa*: El tiempo de ejecución en el peor de los casos crece de manera directamente proporcional a la cantidad total de artículos en la lista. Si hay el doble de elementos y el presupuesto alcanza para evaluarlos, tomará el doble de tiempo.
- b) *Demostración matemática mediante límites*: El algoritmo ejecuta asignaciones iniciales en un tiempo constante c_1 y un bucle `for` que itera, como máximo, n veces. Dentro del bucle, todas las operaciones (evaluar la condición `if`, imprimir, restar al presupuesto e incrementar la cantidad) toman un tiempo constante c_2 . La función de tiempo total en el peor de los casos es:

$$T(n) = c_1 + c_2 \cdot n$$

- c) Para evaluar el orden de complejidad asintótico respecto a la variable principal n , calculamos el límite cuando n tiende a infinito:

$$\lim_{n \rightarrow \infty} \frac{c_1 + c_2 n}{n} = \lim_{n \rightarrow \infty} \left(\frac{c_1}{n} + c_2 \right) = c_2$$

- d) Al obtener una constante estricta mayor que cero (c_2), se demuestra matemáticamente que la función está acotada por una complejidad lineal $O(n)$.

Otra forma de demostrarlo por definición formal se daría siguiendo los siguientes pasos:

Paso 1: Asignación de costos a las operaciones

- c_1 : Tiempo constante para las operaciones iniciales antes del bucle (declaración del arreglo, variables `n`, `presupuesto`, `cantidad`, `i` y el primer `printf`).
- c_2 : Tiempo constante para las comprobaciones y saltos del bucle en cada iteración (evaluación de `i < n`, el incremento `i++` y la condición `if (presupuesto >= precios[i])`).
- c_3 : Tiempo constante para el bloque de código dentro del `if` que se ejecuta en el peor caso (el `printf`, la resta `presupuesto -= precios[i]` y el incremento `cantidad++`).
- c_4 : Tiempo constante para las instrucciones finales fuera del bucle (el último `printf` y el `return 0`).

Paso 2: Construcción de la función de tiempo total $T(n)$

El tiempo total se calcula sumando los costos fuera del bucle y sumando los costos repetitivos en función de n (el tamaño del arreglo):

$$T(n) = c_1 + c_4 + \sum_{i=0}^{n-1} (c_2 + c_3)$$

Dado que las constantes c_2 y c_3 no dependen del iterador i , la sumatoria se resuelve multiplicando por n :

$$T(n) = c_1 + c_4 + n(c_2 + c_3)$$

$$T(n) = (c_2 + c_3) \cdot n + (c_1 + c_4)$$

Para simplificar el álgebra en la demostración, agrupamos las constantes:

Sea $a = c_2 + c_3$ y sea $b = c_1 + c_4$.

La función de tiempo total nos queda como una ecuación lineal:

$$T(n) = a \cdot n + b$$

Paso 3: Demostración por definición estricta (Big O)

La definición formal establece que $T(n) \in O(n)$ si y solo si existen constantes positivas c y n_0 tales que para todo $n \geq n_0$:

$$T(n) \leq c \cdot n$$

Partiendo de nuestra ecuación:

$$T(n) = a \cdot n + b$$

Para asegurar que la cota superior siempre sea positiva, tomamos el valor absoluto de b :

$$T(n) \leq a \cdot n + |b|$$

Sabemos que para cualquier valor de entrada $n \geq 1$, se cumple matemáticamente que $1 \leq n$. Si multiplicamos ambos lados de esta desigualdad por $|b|$, obtenemos $|b| \leq |b| \cdot n$. Sustituimos esto en nuestra inequación para unificar las variables:

$$T(n) \leq a \cdot n + |b| \cdot n$$

Factorizamos el término común n :

$$T(n) \leq (a + |b|) \cdot n$$

Finalmente, elegimos nuestras constantes que satisfacen la definición:

$$c = a + |b|$$

Sea $n_0 = 1$

Con estas constantes elegidas, hemos demostrado matemáticamente que $T(n) \leq c \cdot n$ para todo $n \geq 1$. Esto confirma de manera formal y rigurosa que el algoritmo tiene una complejidad temporal asintótica de $O(n)$.

Complejidad Espacial: $O(1)$.

- *Qué representa:* El espacio de memoria adicional que utiliza el programa se mantiene mínimo y constante a lo largo de su ejecución, independientemente de si la lista tiene 5 o un millón de precios.
- *Demostración:* El algoritmo solo emplea un grupo reducido de variables primitivas (n , $presupuesto$, $cantidad$, i) para llevar el control del ciclo y las acumulaciones. Al no crear nuevas estructuras de datos dinámicas, no instanciar arreglos adicionales ni utilizar llamadas recursivas que consuman espacio en la pila, el consumo de

memoria extra es exactamente un valor constante k , lo que equivale asintóticamente a $O(1)$.

5) Asignación de Ratones a Agujeros (Mice and Holes)

Problema: Este problema plantea que existen n ratones y n agujeros posicionados en diferentes coordenadas de una recta numérica.

Estrategia greedy: El objetivo es asignar cada ratón a un agujero distinto de manera que el tiempo máximo que tarda cualquier ratón en llegar a su agujero sea el mínimo posible. La estrategia voraz consiste en ordenar las posiciones de los ratones y de los agujeros, y emparejar el ratón i -ésimo con el agujero i -ésimo.

Caso:

- Posiciones iniciales de los ratones: {4, -4, 2}
- Posiciones iniciales de los agujeros: {4, 0, 5}

Paso a paso de la ejecución:

1. **Ordenamiento:** El algoritmo ordena ambos arreglos de menor a mayor (usando `qsort`).
 - Ratones ordenados: {-4, 2, 4}
 - Agujeros ordenados: {0, 4, 5}
2. **Emparejamiento y cálculo de distancias (tiempo):**
 - **Pareja 1:** El ratón en -4 va al agujero en 0. Distancia = $|-4 - 0| = 4$. (El tiempo máximo provisional es 4).
 - **Pareja 2:** El ratón en 2 va al agujero en 4. Distancia = $|2 - 4| = 2$. (El tiempo máximo sigue siendo 4, ya que 2 no es mayor que 4).
 - **Pareja 3:** El ratón en 4 va al agujero en 5. Distancia = $|4 - 5| = 1$. (El tiempo máximo sigue siendo 4).

3. Ejecución del programa:

```
El tiempo minimo en el que todos los ratones estaran a salvo es: 4
-----
Process exited after 0.03778 seconds with return value 0
Presione una tecla para continuar . . .
```

Características, Eficiencia e Implementación

- **Eficiencia:** Es muy eficiente, limitada únicamente por el algoritmo de ordenamiento utilizado. La etapa de emparejamiento procesa los datos en un solo recorrido.

- **Facilidad de implementación:** Altamente sencilla. Requiere llamadas a la función de ordenamiento estándar de C (**qsort**) y un bucle lineal para calcular el máximo absoluto de las diferencias.
- **Características particulares:** Se basa en evitar que las trayectorias de los ratones se crucen. Si dos trayectorias se cruzan, el tiempo máximo siempre será igual o mayor que si no se cruzan.

Ventajas, Desventajas y Situaciones de Uso

- **Ventajas:** Garantiza matemáticamente la solución óptima (minimizando el peor caso). No requiere memoria adicional significativa.
- **Desventajas:** Es un algoritmo *offline*; requiere conocer las posiciones de todos los ratones y agujeros antes de comenzar. Si un nuevo ratón o agujero aparece en tiempo real, se debe recalcular todo.
- **Cuándo resulta conveniente:** En logística y asignación de recursos geográficos donde el "costo" es el tiempo máximo de espera, como dirigir flotas de taxis a clientes cercanos o asignar ambulancias a puntos de emergencia, garantizando que el último recurso en llegar tarde lo menos posible.

Análisis Matemático de Complejidad

Complejidad Temporal: $O(n \log n)$, donde n es la cantidad de ratones/agujeros.

- Qué representa:* El tiempo total de ejecución está dominado por el costo de ordenar los elementos; la evaluación de las distancias crece de forma lineal y es opacada por el ordenamiento al escalar la cantidad de datos.
- Demostración matemática mediante límites:* El algoritmo tiene tres componentes principales: ordenar el arreglo de ratones, ordenar el arreglo de agujeros, y un bucle de comparación.

$$T(n) = T_{ord_ratones}(n) + T_{ord_agujeros}(n) + T_{bucle}(n)$$

- Asumiendo un algoritmo de ordenamiento eficiente como Quicksort/Mergesort:

$$T(n) = c_1 n \log n + c_1 n \log n + c_2 n = 2c_1 n \log n + c_2 n$$

- Para evaluar el orden de complejidad, calculamos el límite respecto a $n \log n$ cuando n tiende al infinito:

$$\lim_{n \rightarrow \infty} \frac{2c_1 n \log n + c_2 n}{n \log n} = \lim_{n \rightarrow \infty} \left(2c_1 + \frac{c_2}{\log n} \right) = 2c_1$$

Al obtener una constante estricta mayor que cero, se demuestra que la función crece asintóticamente a un ritmo de $O(n \log n)$.

Complejidad Espacial: $O(1)$ (o $O(\log n)$ dependiendo de **qsort**).

- *Qué representa:* El espacio en memoria extra que utiliza el programa, sin contar los arreglos originales.

- El bucle solo requiere variables primitivas ($O(1)$). Si la implementación en la biblioteca estándar de C usa QuickSort, la pila recursiva requiere $O(\log n)$ de espacio en promedio.

6) Árbol de expansión mínima con Prim.

Problema: dado un grafo conectado, no dirigido y con pesos, conectar **todos** los vértices usando aristas de costo total mínimo y sin ciclos. (Por ejemplo, tender cable entre ciudades con el menor costo.)

Estrategia greedy: El objetivo es conectar todos los vértices del grafo sin formar ciclos, logrando que la suma total de los pesos de las aristas (costo) sea la mínima posible. La estrategia voraz consiste en seleccionar, en cada iteración, el vértice fuera del árbol actual que tenga el menor costo de conexión conocido hacia algún vértice que ya pertenezca al árbol.

Caso:

- Grafo: Está compuesto por 5 vértices (del 0 al 4).
- Conexiones (Matriz de adyacencia): Define los costos entre nodos. Por ejemplo, conectar el nodo 0 con el 1 cuesta 2, y el 0 con el 3 cuesta 6. Un 0 indica que no hay conexión directa o es el mismo nodo.

Paso a paso de la ejecución:

1. **Vértice inicial (Nodo 0):** Se incluye el vértice 0 en el árbol. Su costo para entrar es 0. Se actualizan los costos mínimos conocidos para llegar a sus vecinos: para ir al nodo 1 cuesta 2, y al nodo 3 cuesta 6.
2. **Elección Greedy 1:** De los nodos fuera del árbol, el más barato de alcanzar es el **nodo 1** (costo 2 desde el nodo 0).
 - Se incluye el nodo 1 al árbol (Costo acumulado: 2).
 - Se evalúan sus vecinos y se actualizan costos: ahora se puede llegar al nodo 2 por un costo de 3, y al nodo 4 por un costo de 5. (Para llegar al nodo 3 desde el 1 cuesta 8, pero como ya teníamos una ruta más barata de costo 6 desde el nodo 0, esa no se actualiza).
3. **Elección Greedy 2:** De los nodos fuera del árbol (2, 3 y 4), el más barato de alcanzar es el **nodo 2** (costo 3 desde el nodo 1).
 - Se incluye el nodo 2 al árbol (Costo acumulado: $2 + 3 = 5$).
 - Se evalúan los vecinos del nodo 2. La ruta al nodo 4 cuesta 7, pero ya teníamos una mejor (costo 5 desde el nodo 1), así que no se actualiza.
4. **Elección Greedy 3:** Quedan los nodos 3 (costo 6) y 4 (costo 5). El más barato es el **nodo 4**.
 - Se incluye el nodo 4 al árbol (Costo acumulado: $5 + 5 = 10$).
 - Se evalúan sus vecinos, pero no mejoran ninguna ruta existente hacia los nodos restantes.
5. **Elección Greedy 4:** Solo queda el **nodo 3** (costo 6 desde el nodo 0).

- Se incluye el nodo 3 al árbol (Costo acumulado: $10 + 6 = 16$).
- Ya todos los nodos están en el árbol.

6. Ejecución del programa:

```

Aristas del arbol de expansion minima:
0 - 1 (costo 2)
1 - 2 (costo 3)
0 - 3 (costo 6)
1 - 4 (costo 5)
Costo total = 16

-----
Process exited after 0.03654 seconds with return value 0
Presione una tecla para continuar . . .

```

Características, Eficiencia e Implementación

- **Eficiencia:** Su eficiencia en la versión presentada está determinada por el uso de bucles anidados que recorren secuencialmente todos los vértices. Es razonablemente eficiente para grafos densos, pero decae en rendimiento si el grafo tiene miles de nodos, debido a la búsqueda lineal del costo mínimo en cada iteración.
- **Facilidad de implementación:** La implementación es directa. Utiliza una matriz de adyacencia para representar el grafo y tres arreglos unidimensionales (**costo**, **padre**, **enArbol**) para rastrear el estado y la mejor conexión de cada vértice.
- **Características particulares:** Construye la solución de forma conectada; es decir, el árbol crece como un solo bloque desde un nodo inicial. Para evitar ciclos iterativos infinitos, utiliza un arreglo booleano/entero (**enArbol**) que marca con un "1" a los vértices que ya fueron consolidados en el árbol de expansión mínima.

Ventajas, Desventajas y Situaciones de Uso

- **Ventajas:** Garantiza matemáticamente encontrar la solución global óptima (el árbol de expansión de costo mínimo absoluto) para cualquier grafo conexo no dirigido.
- **Desventajas:** La implementación básica mostrada mediante matriz de adyacencia es ineficiente en términos de memoria y tiempo para grafos muy grandes y dispersos (donde hay pocos caminos).
- **Cuándo resulta conveniente:** Es ideal en proyectos de infraestructura, como el diseño de redes de fibra óptica, el trazado de cables eléctricos entre múltiples ciudades, o la instalación de tuberías de agua, donde es obligatorio interconectar todos los puntos (nodos) utilizando la menor longitud de material (aristas) posible para minimizar los costos de obra.

Análisis Matemático de Complejidad

Complejidad Temporal: $O(V^2)$, donde V es la cantidad de vértices definidos en el grafo.

- *Qué representa:* El tiempo total de ejecución crece de forma cuadrática en relación a la cantidad de nodos. Si el número de vértices se duplica, el tiempo de procesamiento se cuadruplica.
- *Demostración matemática mediante límites:* El algoritmo está estructurado con un bucle **for** principal que itera V veces. Dentro de este bucle, existen dos bucles **for** secuenciales independientes: el primero itera V veces para buscar el vértice con el menor costo, y el segundo también itera V veces para actualizar los costos de los vecinos. Si asumimos que las operaciones internas (comparaciones y asignaciones) toman un tiempo constante c , la función de tiempo es

$$T(V) = V \cdot (c_1V + c_2V) = (c_1 + c_2)V^2$$

- Para demostrar la complejidad asintótica, calculamos el límite respecto a V^2 cuando V tiende a infinito:

$$\lim_{V \rightarrow \infty} \frac{(c_1 + c_2)V^2}{V^2} = c_1 + c_2$$

- Al obtener una constante estricta mayor que cero, se demuestra asintóticamente que el algoritmo opera a un ritmo de $O(V^2)$.

También se pudo haber demostrado por definición formal, en la que se aplican desigualdades, pero al final terminaría siendo un poco más engorroso para el lector; por ello, es mucho más elegante y directo hacer esta demostración por medio de límites.

Complejidad Espacial: $O(V^2)$ (o $O(V)$ de espacio auxiliar).

- *Qué representa:* El espacio en memoria requerido para almacenar la estructura del problema y las variables de control.
- *Demostración:* La estructura principal es la matriz de adyacencia bidimensional **grafo[V][V]**, la cual requiere $V \times V$ espacios de memoria. Adicionalmente, se crean tres arreglos unidimensionales (**costo**, **padre**, **enArbol**), cada uno de tamaño V . El espacio total es $S(V) = V^2 + 3V$. Como el término dominante es V^2 , la complejidad de almacenamiento total es $O(V^2)$. Si se evaluara únicamente el espacio extra utilizado sin contar los datos de entrada, la complejidad espacial auxiliar sería $O(V)$ por los tres arreglos unidimensionales.

7) Policías y Ladrones (Police and thieves)

Problema: Se tiene un arreglo unidimensional donde cada elemento es 'P' (policía) o 'T' (ladrón). Se da un entero K que representa la distancia máxima a la que un policía puede atrapar a un ladrón. Un policía puede atrapar a un solo ladrón y un ladrón puede ser atrapado por un solo policía. El objetivo es maximizar la cantidad de ladrones atrapados.

Datos de prueba.

Caso 1: arr = [P, T, T, P, T], $K=1$.

Recorrido de izquierda a derecha:

$i=0$: Encuentra 'P'. Cola de P = [0].

i=1: Encuentra 'T'. Cola de T = [1]. Hay P y T. Comparan: $|0 - 1| = 1$. Como $1 \leq K(1)$, ¡Captura! Contador = 1. Colas vacías.
i=2: Encuentra 'T'. Cola de T = [2].
i=3: Encuentra 'P'. Cola de P = [3]. Hay P y T. Comparan: $|3 - 2| = 1$. Como $1 \leq K(1)$, ¡Captura! Contador = 2. Colas vacías.
i=4: Encuentra 'T'. Cola de T = [4]. Fin del arreglo. No hay policías para emparejar.

Resultado final: 2 capturas.

Caso 2: arr = [T, P, T, P], K=2.

i=0: 'T' -> Cola T = [0]
i=1: 'P' -> Cola P = [1]. Comparan: $|1 - 0| = 1 \leq 2$. ¡Captura! (Contador=1). Colas vacías.
i=2: 'T' -> Cola T = [2]
i=3: 'P' -> Cola P = [3]. Comparan: $|3 - 2| = 1 \leq 2$. ¡Captura! (Contador=2). Colas vacías.

Resultado final: 2 capturas.

Caso 3: arr = [P, T, P, T], K=0.

i=0: 'P' -> Cola P = [0]
i=1: 'T' -> Cola T = [1]. Comparan: $|1 - 0| = 1 > 0$. No captura. Descartan al policía (frenteP++).
i=2: 'P' -> Cola P = [2] (frente en 2). Cola T tiene [1]. Comparan: $|2 - 1| = 1 > 0$. No captura. Descartan al ladrón (frenteT++).
i=3: 'T' -> Cola T = [3]. Cola P tiene [2]. Comparan: $|3 - 2| = 1 > 0$. No captura. Descartan al policía.

Resultado final: 0 capturas.

Ejecución del programa:

```
Caso 1: 2 capturas
Caso 2: 2 capturas
Caso 3: 0 capturas

-----
Process exited after 0.9162 seconds with return value 0
Press any key to continue . . . |
```

Algoritmo greedy: Colas de índices

Estrategia voraz:

Recorrer el arreglo de izquierda a derecha. Mantener dos colas (o listas) con los índices de los policías y ladrones encontrados hasta el momento. Cuando ambas colas no estén vacías, comparar sus frentes:

1. Si $|i - j| \leq K$, se realiza la captura, se incrementa el contador y se eliminan ambos frentes.

2. Si $i < j$ y $j - i > K$, el policía en i ya no puede alcanzar a ningún ladrón futuro (los siguientes estarán más lejos), así que se descarta.
3. Si $j < i$ y $i - j > K$, se descarta el ladrón en j .

Este proceso garantiza que siempre se empareja al policía y al ladrón más antiguos que estén dentro del rango K , evitando desperdiciar capturas.

Características, eficiencia e implementación

- Eficiencia: Un solo recorrido del arreglo. Las operaciones de encolar y desencolar son $O(1)$. El costo total es lineal.
- Facilidad de implementación: sencilla. Se usan dos arreglos auxiliares que actúan como colas, con índices de frente y final.
- Características particulares: La estrategia es óptima porque, al procesar de izquierda a derecha, cualquier policía que no pueda atrapar al ladrón más antiguo dentro de K tampoco podrá atrapar a los ladrones posteriores (estarán aún más lejos). Por lo tanto, descartarlo es seguro.

Ventajas, desventajas y situaciones de uso

- Ventajas: Encuentra siempre la solución óptima, es rápido y no requiere ordenamiento previo.
- Desventajas: Utiliza memoria adicional $O(n)$ en el peor caso (cuando todos los elementos son policías o todos son ladrones).
- Cuándo resulta conveniente: Problemas de asignación por cercanía en una línea, como asignar taxis a clientes en una avenida, emparejar drones de reparto con puntos de entrega, o asignar recursos limitados a solicitudes con restricciones de distancia.

Análisis matemático de complejidad

Complejidad temporal: $O(n)$, donde n es la longitud del arreglo.

- ¿Qué representa? El tiempo de ejecución crece de forma directamente proporcional a la cantidad de elementos del arreglo. Si se duplica el número de policías y ladrones, el tiempo también se duplica aproximadamente.
- Demostración mediante límites: El algoritmo recorre el arreglo una sola vez. Dentro del ciclo, cada elemento se encola como máximo una vez y se desencola como máximo una vez. Las operaciones internas toman un tiempo constante c . La función de tiempo es:

$$T(n) = c \cdot n$$

Calculamos el límite cuando $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} \frac{T(n)}{n} = c > 0$$

Al obtener una constante finita y positiva, se demuestra que $T(n)$ es de orden $O(n)$.

Complejidad espacial: $O(n)$ en el peor caso.

- ¿Qué representa? La memoria adicional utilizada por las colas que almacenan los índices de policías y ladrones no emparejados.

- Demostración: En el peor caso, todos los elementos son del mismo tipo (por ejemplo, todos policías). Entonces, una de las colas almacenará hasta n índices. El espacio auxiliar es proporcional a n . Si se considera que K es pequeño y los elementos no emparejados expiran rápidamente, el espacio puede acotarse a $O(K)$, pero en el peor caso K puede ser n , por lo que la cota es $O(n)$.

8) Programación de trabajos con fechas límite (Job Sequencing with Deadlines)

Problema: Se tienen n trabajos. Cada trabajo i tiene una fecha límite d_i y un beneficio p_i si se completa antes de dicha fecha. Cada trabajo tarda exactamente 1 unidad de tiempo y solo se puede ejecutar un trabajo a la vez. El objetivo es maximizar el beneficio total seleccionando un subconjunto de trabajos que puedan completarse antes de sus respectivas fechas límite.

Datos de prueba.

Caso 1: J1(d_2, p_{100}), J2(d_1, p_{50}), J3(d_1, p_{30})

Ordenados por beneficio: J1(100), J2(50), J3(30). Deadline máximo = 2. Arreglo slots = [0, 0, 0].

J1 ($d=2, p=100$): Busca el último slot libre ≤ 2 . El slot 2 está libre. Se asigna. slots = [0, 0, J1]. Beneficio = 100.

J2 ($d=1, p=50$): Busca el último slot libre ≤ 1 . El slot 1 está libre. Se asigna. slots = [0, J2, J1]. Beneficio = 150.

J3 ($d=1, p=30$): Busca el último slot libre ≤ 1 . El slot 1 está ocupado. No hay slots libres. Se descarta.

Resultado final: Beneficio total = 150. (Se seleccionan J1 y J2).

Caso 2: J1(d_3, p_{40}), J2(d_1, p_{20}), J3(d_2, p_{60}), J4(d_2, p_{30})

Ordenados por beneficio: J3(60), J1(40), J4(30), J2(20). Deadline máximo = 3. slots = [0, 0, 0, 0].

J3 ($d=2, p=60$): Último slot libre ≤ 2 es el 2. Se asigna. slots = [0, 0, J3, 0]. Beneficio = 60.

J1 ($d=3, p=40$): Último slot libre ≤ 3 es el 3. Se asigna. slots = [0, 0, J3, J1]. Beneficio = 100.

J4 ($d=2, p=30$): Último slot libre ≤ 2 es el 1. Se asigna. slots = [0, J4, J3, J1]. Beneficio = 130.

J2 ($d=1, p=20$): Último slot libre ≤ 1 es ninguno (el 1 está ocupado). Se descarta.

Resultado final: Beneficio total = 130. (Se seleccionan J1, J3 y J4). El algoritmo incluye J4 porque aporta 30 sin desplazar a ningún trabajo más rentable.

Ejecución de programa:

```
=== Caso 1 ===
Trabajos seleccionados:
  J1 (deadline=2, beneficio=100)
  J2 (deadline=1, beneficio=50)
Beneficio total = 150

=== Caso 2 ===
Trabajos seleccionados:
  J3 (deadline=2, beneficio=60)
  J1 (deadline=3, beneficio=40)
  J4 (deadline=2, beneficio=30)
Beneficio total = 130

-----
Process exited after 0.9063 seconds with return value 0
Press any key to continue . . . |
```

Algoritmo greedy: Beneficio decreciente con asignación al último slot libre

Estrategia

voraz:

1. Ordenar todos los trabajos en orden descendente según su beneficio p_i .
2. Recorrer los trabajos ordenados. Para cada trabajo, buscar el último slot de tiempo libre que sea menor o igual a su fecha límite d_i .
3. Si existe un slot disponible, asignar el trabajo a ese slot y sumar su beneficio al total. Si no existe, descartar el trabajo.

La idea clave es que, al elegir primero los trabajos más rentables, aseguramos que no se pierdan las ganancias mayores. Asignar cada trabajo al último slot disponible antes de su deadline preserva los slots más tempranos para trabajos que puedan tener fechas límite más estrictas, maximizando así la cantidad total de trabajos realizables.

Características, eficiencia e implementación

- Eficiencia: El ordenamiento inicial domina el costo. La búsqueda del slot libre para cada trabajo puede requerir, en el peor caso, recorrer todos los slots disponibles.
- Facilidad de implementación: sencilla. Se utiliza un arreglo de estructuras para los trabajos y un arreglo de enteros slot [] de tamaño igual al máximo deadline, donde $slot[t] = 1$ indica que el tiempo t está ocupado.
- Características particulares: A lo sumo se realizan n asignaciones. El algoritmo es óptimo porque en cada paso toma la decisión más rentable que aún es factible, y el problema restante conserva la misma estructura (subestructura óptima).

Ventajas, desventajas y situaciones de uso

- Ventajas: Encuentra siempre la solución óptima. Es intuitivo y fácil de programar. No requiere estructuras de datos complejas en su versión básica.

- Desventajas: La implementación básica tiene complejidad $O(n^2)$ debido a la búsqueda lineal del slot libre. Puede optimizarse a $O(n \log n)$ usando una estructura de conjuntos disjuntos (Union-Find), pero se mantiene el análisis básico para este nivel.
- Cuándo resulta conveniente: Planificación de tareas con plazos y prioridades, como trabajos de impresión, procesos por lotes, asignación de tareas a un procesador único, o selección de proyectos con fechas de entrega y ganancias asociadas.

Análisis matemático de complejidad

Complejidad Temporal: $O(n^2)$ en el peor caso.

- ¿Qué representa? El tiempo de ejecución crece de forma cuadrática con la cantidad de trabajos. Si se duplica el número de trabajos, el tiempo se multiplica aproximadamente por cuatro.
- Demostración mediante límites: El ordenamiento toma $c_1 n \log n$. La búsqueda del slot libre para cada trabajo puede requerir hasta $O(n)$ comparaciones. En el peor caso, la suma de comparaciones es $\frac{n(n-1)}{2}$.

Las operaciones de asignación y suma toman $c_3 n$.

$$T(n) = c_1 n \log n + c_2 \frac{n(n-1)}{2} + c_3 n$$

Calculamos el límite cuando $n \rightarrow \infty$ respecto a n^2 :

$$\lim_{n \rightarrow \infty} \frac{T(n)}{n^2} = \lim_{n \rightarrow \infty} \left(\frac{c_1 \log n}{n} + \frac{c_2}{2} \cdot \frac{n-1}{n} + \frac{c_3}{n} \right) = \frac{c_2}{2}$$

Como el límite es una constante finita y mayor que cero, se demuestra que $T(n)$ es de orden $O(n^2)$.

Complejidad Espacial: $O(n)$ (o $O(\max(d_i))$).

- ¿Qué representa la memoria adicional utilizada por el arreglo de slots, cuyo tamaño depende de la fecha límite máxima? En el peor caso, $\max(d_i) = n$, por lo que el espacio es lineal.
- Demostración: Se declara un arreglo slot de tamaño $\max(d_i) + 1$. Como $\max(d_i) \leq n$ en la mayoría de los casos prácticos (o puede acotarse a n), el espacio auxiliar es $O(n)$. Las demás variables son de tamaño constante.

9) Conexión de Cuerdas con Costo Mínimo (Optimal Merge Pattern)

Problema: Se tienen n cuerdas de longitudes $l_1, l_2, \dots, l_n$. Se desea unir las todas en una sola cuerda. El costo de unir dos cuerdas es la suma de sus longitudes. El objetivo es minimizar el costo total de todas las uniones.

Datos de prueba.

Caso 1: cuerdas = [1,2,3,4,5].

Min-Heap inicial: [1, 2, 3, 4, 5]. Costo Total = 0.

Iteración 1: Extrae los dos menores (1 y 2). Suma = 3. Costo Total = 3. Inserta 3. Heap actual: [3, 3, 4, 5].

Iteración 2: Extrae los dos menores (3 y 3). Suma = 6. Costo Total = 3 + 6 = 9. Inserta 6. Heap actual: [4, 5, 6].

Iteración 3: Extrae los dos menores (4 y 5). Suma = 9. Costo Total = 9 + 9 = 18. Inserta 9. Heap actual: [6, 9].

Iteración 4: Extrae los dos menores (6 y 9). Suma = 15. Costo Total = 18 + 15 = 33. Inserta 15. Heap actual: [15].

Fin: Solo queda un elemento en el heap. Se detiene.

Resultado final: Costo mínimo = 33.

Caso 2: cuerdas = [1,2,3,100].

Min-Heap inicial: [1, 2, 3, 100]. Costo Total = 0.

Iteración 1: Extrae 1 y 2. Suma = 3. Costo Total = 3. Inserta 3. Heap actual: [3, 3, 100].

Iteración 2: Extrae 3 y 3. Suma = 6. Costo Total = 3 + 6 = 9. Inserta 6. Heap actual: [6, 100].

Iteración 3: Extrae 6 y 100. Suma = 106. Costo Total = 9 + 106 = 115. Inserta 106. Heap actual: [106].

Fin: Solo queda un elemento.

Resultado final: Costo mínimo = 115.

Ejecución de programa:

```
Caso 1: costo minimo = 33
Caso 2: costo minimo = 115

-----
Process exited after 0.0118 seconds with return value 0
Press any key to continue . . . |
```

Algoritmo greedy: Cola de prioridad mínima (min-heap)

Estrategia voraz:

1. Insertar todas las longitudes de las cuerdas en un min-heap (cola de prioridad mínima).
2. Mientras haya más de una cuerda en el heap:
 - a. Extraer las dos cuerdas más pequeñas, a y b.
 - b. Sumar su longitud a +b y acumular ese valor al costo total.
 - c. Insertar la nueva cuerda a+b de nuevo en el heap.
3. Cuando solo quede una cuerda, el costo acumulado es el mínimo posible.

La idea clave es unir siempre las dos cuerdas más cortas disponibles. Así, las cuerdas largas se van formando al final y se suman menos veces al costo total, minimizando el impacto de las longitudes grandes.

Características, eficiencia e implementación

- Eficiencia: Cada operación de extracción o inserción en el heap toma $O(\log n)$. Se realizan $n-1$ uniones, por lo que el costo total es $O(n \log n)$.
- Facilidad de implementación: Requiere implementar un min-heap (arreglo con funciones heapify, extractMin e insert). Es sencillo y no necesita ordenar previamente todo el arreglo.
- Características particulares: El algoritmo es óptimo porque el argumento es de intercambio: en una unión óptima, las dos cuerdas más cortas pueden suponerse unidas primero sin aumentar el costo. La subestructura es $\text{costo}(L) = (a+b) + \text{costo}(L')$, con L' la lista con $a+b$ en lugar de a y b . Es la misma idea de Huffman.

Ventajas, desventajas y situaciones de uso

- Ventajas: Encuentra siempre la solución óptima. Es eficiente incluso para n grande. No requiere ordenamiento previo completo.
- Desventajas: Necesita memoria adicional para el heap ($O(n)$). Si no se implementa bien el heap, puede degradarse a $O(n^2)$.
- Cuándo resulta conveniente: Compresión de datos (Huffman), fusión de archivos, planificación de ensamblajes, unión de listas ordenadas, y en general cualquier problema donde se deban combinar elementos con costo acumulativo.

Análisis matemático de complejidad

Complejidad Temporal: $O(n \log n)$.

- ¿Qué representa? El tiempo de ejecución crece ligeramente más rápido que linealmente, debido a las operaciones de heap. Si se duplica el número de cuerdas, el tiempo se duplica y se añade un factor logarítmico.
- Demostración mediante límites: Se realizan $n-1$ extracciones dobles y $n-1$ inserciones. Cada operación de heap cuesta $c_1 \log n$. Las operaciones constantes adicionales suman $c_2 n$.

La función de tiempo es:

$$T(n) = c_1 n \log n + c_2 n$$

Calculamos el límite cuando $n \rightarrow \infty$ respecto a $n \log n$:

$$\lim_{n \rightarrow \infty} \frac{T(n)}{n \log n} = \lim_{n \rightarrow \infty} \left(c_1 + \frac{c_2}{\log n} \right) = c_1$$

Como el límite es una constante finita y mayor que cero, se demuestra que $T(n)$ es de orden $O(n \log n)$.

Complejidad Espacial: $O(n)$.

- ¿Qué representa? La memoria adicional utilizada por el min-heap, que almacena hasta n elementos.
- Demostración: El heap se implementa como un arreglo de tamaño n . No se crean otras estructuras que dependan de n . Por lo tanto, el espacio auxiliar es lineal.

BIBLIOGRAFÍA:

- **CP-Algorithms.** (s.f.). *Competitive Programming Algorithms: Algoritmos de Grafos y Matemáticas.* e-maxx.ru. Recuperado de: <https://cp-algorithms.com/>
- **Coursera / UC San Diego.** (s.f.). *Algorithmic Toolbox: Greedy Algorithms.* Recuperado de: <https://www.coursera.org/learn/algorithmic-toolbox>
- **GeeksforGeeks.** (s.f.). *Greedy Algorithms: Tutorials and Examples.* Recuperado de: <https://www.geeksforgeeks.org/greedy-algorithms/>
- **HackerEarth.** (s.f.). *Basics of Greedy Algorithms: Tutorials & Practice.* Recuperado de: <https://www.hackerearth.com/practice/algorithms/greedy/basics-of-greedy-algorithms/tutorial/>
- **Khan Academy.** (s.f.). *Ciencias de la computación: Algoritmos.* Recuperado de: <https://es.khanacademy.org/computing/computer-science/algorithms>
- **LeetCode.** (s.f.). *Greedy Tag: Problems and Editorial Discussions.* Recuperado de: <https://leetcode.com/tag/greedy/>
- **MIT OpenCourseWare.** (2015). *Design and Analysis of Algorithms (6.046J/18.410J).* Massachusetts Institute of Technology. Recuperado de: <https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/>